



NB :

- Le dossier de ressources associé à cet exercice est accessible à l'adresse suivante :

<https://shorturl.at/V4Hor>

- Il vous est conseillé d'ajouter des type hints aux fonctions et variables que vous créez dans ces exercices. N'oubliez pas non plus de rédiger les docstrings de vos fonctions.

Exercice 1 : Analyse de chaîne de caractères

L'objectif de cet exercice est d'écrire un programme en Python qui permet d'analyser une chaîne de caractères donnée par l'utilisateur.

Le programme devra compter le nombre d'apparitions de chaque caractère alphabétique présent dans la chaîne, en ignorant les caractères non alphabétiques et en ne distinguant pas les majuscules des minuscules.

Enfin, il affichera les résultats sous forme d'une liste avec chaque lettre et son nombre d'occurrences.

Exemple de trace d'exécution :

"Bonjour, ceci est un test !"

Sortie

```
b: 1  
o: 2  
n: 2  
j: 1  
u: 2  
r: 1  
c: 1  
e: 3  
i: 2  
s: 2  
t: 3
```

Écrire le programme réalisant l'objectif cité plus haut en utilisant exclusivement :

- Les listes comme structures de données.
- Les dictionnaires comme structures de données.

Exercice 2 : Mesure de température

NB : Le fichier "observations.txt" est disponible dans le dossier «ressources.zip» joint à cet exercice.

Les chercheurs étudiant les changements climatiques réalisent des observations dans certains jours de l'année.

Les degrés de température proviennent de capteurs. Une observation sera une liste de 3 valeurs [jour, mois, degré].

Par exemple, une température de 20° C enregistrée le 13 février correspond à la liste [13, 2, 20].

Pour simplifier, on supposera que toutes les observations sont enregistrées pendant des années non bissextiles (le nombre de jours du mois de février d'une année non bissextile est égale à 28).

On suppose que les listes `l_jours` et `l_mois` suivantes sont préalablement définies :

```
1. # Liste des jours par mois (non bissextile).
2. l_jours = [31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31]
3. # Liste des mois en français.
4. l_mois = ['Janvier', 'Février', 'Mars',
5.           'Avril', 'Mai', 'Juin', 'Juillet',
6.           'Août', 'Septembre', 'Octobre',
7.           'Novembre', 'Décembre']
```

1. Écrire une fonction **dict_mois_jours** qui, à partir des deux listes `l_jours` et `l_mois`, retourne un dictionnaire associant chaque mois à son nombre de jours.
2. Écrire une fonction **dict_ord_mois_jours** retournant un dictionnaire qui associe à chaque mois de la liste `mois`, son numéro dans l'année.
3. Écrire une fonction `observation_valide` qui vérifie si une observation est valide.

Une observation est valide si elle contient exactement 3 éléments :

- Le jour (1 à 31 selon le mois)
- Le mois (1 à 12)
- Le degré (entre -50 et 50)

1. Écrire une fonction **decrite_observation** qui retourne une description textuelle d'une observation. Par exemple [14, 2, 10] donnera "Observation : 14 février, Température : 10 °C"
2. Écrire une fonction `importer_observations` qui permet d'importer les observations depuis le fichier "observations.txt". Chaque ligne du fichier est au format : "jour mois degré" et retourne une liste d'observations valides.
3. Afficher la description textuelle de toutes les observations importées.
4. Écrire une fonction `numero_jour` qui retourne le numéro du jour dans l'année pour une observation. Par exemple, l'observation [14, 2, 10] correspond au 45e jour de l'an (14 février).
5. Écrire une fonction `avant` qui prend en paramètre deux observations `obs1` et `obs2` et qui retourne `True` si `obs1` est mesuré avant `obs2`, `False` sinon.
6. Écrire une fonction **apres** qui prend en paramètre deux observations `obs1` et `obs2` et qui retourne `True` si `obs1` est mesuré après `obs2`, `False` sinon.

7. Écrire une fonction nommée `moyenne_temp` qui calcule la température moyenne des observations valides.
8. Écrire une fonction **`compter_observations_par_mois`** qui retourne un dictionnaire associant chaque mois avec son nombre d'observations valides.
9. Implémenter les fonctions `mois_plus_froid` et `mois_plus_chaud` permettant respectivement de trouver le mois avec la température la plus froide et celle avec la température la plus chaude.
10. Sauvegarde dans un fichier texte chaque mois avec la moyenne de ses températures. Chaque ligne du fichier sera au format : "mois température_moyenne".

Exercice 3 : Notes et bonus

NB : Le fichier "bonus.csv" est disponible dans le dossier «ressources.zip» joint à cet exercice.

On dispose d'un fichier sous format CSV (Comma Separated Values – valeurs séparées par des virgules) qui contient des informations concernant les résultats d'un test intermédiaire, c'est-à-dire qu'il y a plusieurs lignes du type :

```
prenom,nom,faculte,note,bonus
```

Par exemple :

```
1. Eric,Clark,economie,13.5,1.0
2. Lisa,Clark,droit,8.0,0.0
```

1. Écrivez une fonction lecture qui prend en paramètres le nom du fichier, ouvre le fichier, le lit et renvoie un dictionnaire contenant les données du fichier.

Les clés du dictionnaire devront correspondre au prénom et nom complet de la personne et leurs valeurs devront correspondre aux données complètes.

Par exemple, les données de l'exemple précédent seront converties comme suit :

```
1. donnees = {
2.   'Lisa Clark': ['Lisa', 'Clark', 'droit', '8.0', '0.0'],
3.   'Eric Clark': ['Eric', 'Clark', 'economie', '13.5', '1.0']
4. }
```

1. Écrivez une deuxième fonction qui prend en paramètres la liste des données d'une personne et les affiche de manière lisible.
2. Écrivez une troisième fonction qui va afficher les résultats du test pour les étudiants ayant obtenu une note au-dessus de 9.5. Elle doit prendre deux paramètres : les données sous la forme d'un dictionnaire (cf. point 1) et la note au-dessus de laquelle il faut afficher l'information, par défaut.
3. Tester vos fonctions en affichant des données du fichier associé à cet exercice.

Exercice 4 : Le codage morse

Le codage Morse est un système de communication inventé dans les années 1830 par Samuel Morse et Alfred Vail. Il utilise des points (·) et des tirets (–) pour représenter les lettres, chiffres et symboles.

Par exemple, la lettre E est codée par un simple point (·), tandis que T est un tiret (-). Son utilité principale était de transmettre des messages à distance via le télégraphe, la radio ou des signaux lumineux, notamment en mer, en aviation ou en situation d'urgence.

L'image ci-contre représente la correspondance entre les lettres de l'alphabet et leurs codes morse.

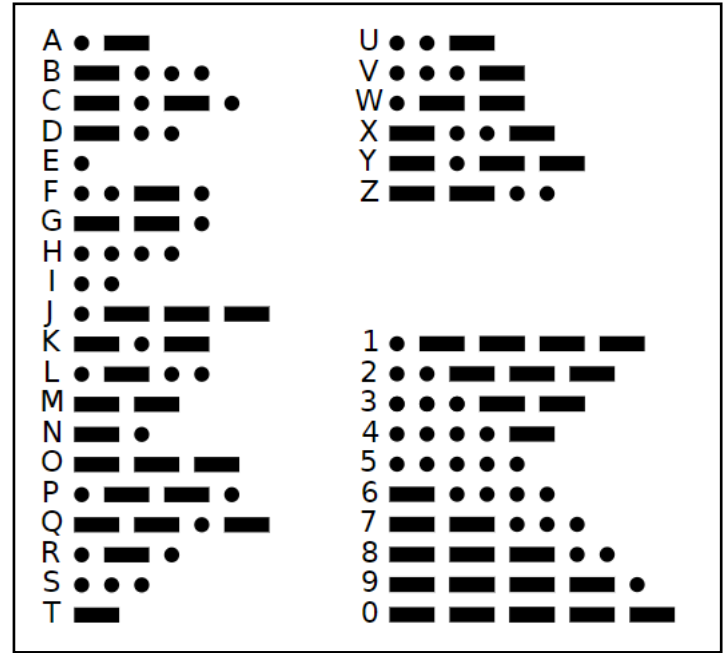


Fig. 1. – Alphabet morse

On souhaite représenter ces informations à l'aide des dictionnaires en python.

1. Construire le dictionnaire appelé morse associant chaque caractère (en clé) à son code morse (en valeur).
2. Écrire une fonction **encode_mot** prenant en paramètre une chaîne de caractères et renvoyant le code morse correspondant sous la forme d'une chaîne de caractères. Les lettres encodées doivent être espacées avec le symbole ".".
3. Pourquoi est-il compliqué (pour l'instant) d'écrire une fonction decode_mot qui ferait l'inverse et décode une chaîne de caractères en morse vers une chaîne de caractères alphabétiques ?
4. Écrire une fonction **inverse** prenant en paramètre le dictionnaire morse précédent et renvoyant son inverse, c'est-à-dire le dictionnaire renvoyant chaque code morse à une lettre. Utilisez la fonction items des dictionnaires.
5. En déduire une fonction **decode_mot** prenant en paramètre un mot en code morse et renvoyant le mot correspondant.
6. Écrire la fonction **decode_phrase** qui prend en paramètre une phrase entière en code morse et la décode. Les mots de la phrase sont séparés par le symbole "/" entouré d'espaces.
7. Quel message cache le code morse suivant ?

[illegible]

Exercice 5 : Matches

Le dictionnaire matches suivant regroupe pour plusieurs clubs de rugby la liste des résultats des derniers matches, avec “V” pour victoire (**4 pts**), “N” pour match nul (**2 pts**) et “D” pour défaite (**1 pt**). Le score d’un club au championnat est donné par la somme des points obtenus dans les matches qu’il a joués.

```
1. matches = {  
2. 'DYTO': ['V', 'V', 'N', 'D', 'D', 'V'],  
3. 'ETOILE FILANTE': ['N', 'D', 'N', 'V', 'V', 'D', 'V'],  
4. 'AS Togo Port': ['D', 'D', 'N', 'D', 'V'],  
5. 'ASKo Kara': ['V', 'V', 'V', 'V', 'D', 'V'],  
6. 'GOMIDO FC': ['V', 'D', 'V', 'D', 'D']  
7. }
```

1. Écrivez un programme qui construit le dictionnaire `scores` dont les clés sont les clubs et les valeurs sont les scores au championnat en chargeant les informations depuis le fichier "matches.csv".
2. Comme tous les clubs n'ont pas joué le même nombre de matches, écrivez le programme qui construit le nouveau dictionnaire **average** qui associe à chaque club son score moyen.
3. Faites un classement des clubs.